

PRODUCT REQUIREMENTS DOCUMENT

Agentic Data Analytics Platform for Engineering Datasets

Conversational Analytics for Engineering Datasets

Version	1.0
Status	Draft
Date	May 31, 2026
Product	Agentic Data Analytics Platform for Engineering Datasets
Audience	Engineering, Design, Product

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Product Vision	3
1.3	Scope	3
1.4	Definitions	3
2	Problem Statement	4
3	Target Users	4
3.1	Primary Persona: Engineering Analyst	4
4	Core Concepts	4
4.1	Project Workspace	5
4.2	Design Principles	5
5	Functional Requirements	5
5.1	Authentication & User Management	5
5.2	Project Management	5
5.3	Dataset Ingestion & Management	5
5.4	Conversational Analytics	6
5.5	Multi-Agent Architecture	6
5.5.1	Planner Agent	6
5.5.2	Data Agent	6
5.5.3	Visualization Agent	6
5.5.4	Insight Agent	7
5.6	Artifact System	7
5.7	Execution Trace	7
5.8	SQL Visibility & Editability	7
5.9	Memory Architecture	7
5.10	User Journey	8
6	Non-Functional Requirements	8
7	Technology Stack	9
7.1	Storage Architecture Rationale	9
8	Data Model	9
9	Tool Calling Architecture	10
10	Goals & Non-Goals	10
10.1	Goals (MVP)	10
10.2	Non-Goals (Explicitly Excluded from MVP)	11
11	Key Differentiators	11

12 Success Criteria	11
13 Future Expansion (Out of Scope)	12
Document History	12

1 Introduction

1.1 Purpose

This Product Requirements Document (PRD) defines the functional and non-functional requirements for the **Agentic Data Analytics Platform for Engineering Datasets** — a conversational, multi-agent data analytics platform purpose-built for engineering datasets. It serves as the authoritative reference for engineering, design, and product stakeholders throughout the development lifecycle.

1.2 Product Vision

Vision Statement

“ChatGPT for Data Analysis” — a persistent, conversational workspace where engineering analysts can explore complex datasets through natural language, powered by a multi-agent AI system that accumulates knowledge over time.

The product is positioned as:

- **Vibe Analytics** — low-friction, curiosity-driven data exploration
- **Conversational Analytics** — natural language as the primary interface
- **Persistent Knowledge Workspace** — not a temporary chat session
- **Multi-Agent System** — specialised agents, not a single AI assistant
- **Analytics-first** — not a BI dashboard builder

1.3 Scope

This PRD covers the Minimum Viable Product (MVP) of the platform. Future capabilities are explicitly listed in Section 10 as non-goals.

1.4 Definitions

Term	Definition
Artifact	A persistent, agent-generated analytical output (e.g. a monthly defect summary table).
Workspace	A Project containing one Conversation, multiple Datasets, Artifacts, Charts, and Execution Traces.
Planner Agent	The orchestrating agent responsible for task planning, re-planning, and agent coordination.
Data Agent	The agent responsible for schema understanding, SQL generation, and query execution.
Viz Agent	The Visualization Agent that recommends and renders charts from result tables.
Insight Agent	The agent that detects trends and anomalies and synthesises observations.

Term	Definition
Execution Trace	A user-visible log of all agent actions taken during a query resolution.
DuckDB	The embedded analytical SQL engine used for querying Parquet files.

2 Problem Statement

Engineering analysts face significant friction when working with multiple datasets:

- Data is scattered across many CSV/XLSX files with unknown relationships.
- Cross-dataset analysis requires manual joins and deep schema familiarity.
- Traditional analytics workflows demand SQL expertise that most analysts lack.
- Existing tools create a gap between *curiosity* and *insight*.
- Analysis results are ephemeral — difficult to reuse or rediscover.
- There is no mechanism for the system to remember previous analyses.

3 Target Users

3.1 Primary Persona: Engineering Analyst

Persona — Engineering Analyst

Goals	Explore engineering datasets (production, quality, testing, sensor data) quickly; perform cross-dataset analysis; generate visualisations; discover anomalies and trends — all <i>without writing SQL</i> .
Pain Points	Data scattered across files; unknown dataset relationships; repetitive analysis workflows; manual SQL writing; inability to reuse previous results.
Skills	Engineering domain knowledge — High ; Excel — High ; SQL — Low ; Programming — Low .

4 Core Concepts

4.1 Project Workspace

Each user owns one or more **Projects**. A Project is the top-level container that forms an *analytical journey*:

User → Project → {Datasets, Artifacts,
Charts, Execution Traces, Conversation}

4.2 Design Principles

1. Users should think about **questions**, not SQL, tables, or schemas.
2. Analytics should feel **conversational** — the ideal input is a natural language question.
3. The system should **accumulate knowledge** — every interaction enriches project memory.
4. **Artifacts** are more valuable than chat messages — they represent reusable knowledge.
5. Execution should be **transparent** — users must understand what happened and why, without exposing chain-of-thought reasoning.

5 Functional Requirements

5.1 Authentication & User Management

- **FR-AUTH-01:** The system shall support user registration with email and password.
- **FR-AUTH-02:** The system shall support local email/password login.
- **FR-AUTH-03:** OAuth and SSO are explicitly *out of scope* for MVP.

5.2 Project Management

- **FR-PROJ-01:** A user shall be able to create, view, and manage multiple Projects.
- **FR-PROJ-02:** Each Project shall contain exactly *one* Conversation (not multiple).
- **FR-PROJ-03:** Each Project shall contain Datasets, Artifacts, Charts, and Execution Traces.

5.3 Dataset Ingestion & Management

- **FR-DS-01:** Users shall be able to upload datasets in CSV or XLSX format.
- **FR-DS-02:** Upon upload, the system shall automatically profile datasets and extract column metadata.
- **FR-DS-03:** Uploaded files shall be converted to Parquet format and stored in S3-compatible object storage.

- **FR-DS-04:** Datasets shall be **read-only** — editing and deletion are not permitted.
- **FR-DS-05:** Users shall be able to view dataset schemas and profiling results.

5.4 Conversational Analytics

- **FR-CONV-01:** Users shall be able to ask questions in natural language within the Project Conversation.
- **FR-CONV-02:** The system shall parse user intent and produce an execution plan before querying data.
- **FR-CONV-03:** The system shall support follow-up questions that build on prior context.
- **FR-CONV-04:** The system shall support cross-dataset analysis (joining multiple datasets).

5.5 Multi-Agent Architecture

The platform employs four specialised agents coordinated by the Planner Agent.

5.5.1 Planner Agent

- **FR-PLAN-01:** Shall create execution plans from user queries.
- **FR-PLAN-02:** Shall observe agent failures and re-plan workflows autonomously.
- **FR-PLAN-03:** Shall ask the user for clarification when intent is ambiguous.
- **FR-PLAN-04:** Shall retrieve relevant Artifacts from memory to augment planning.
- **FR-PLAN-05:** Shall coordinate all downstream agents and manage the execution lifecycle.

5.5.2 Data Agent

- **FR-DATA-01:** Shall understand dataset schemas and infer inter-dataset relationships at runtime.
- **FR-DATA-02:** Shall generate DuckDB SQL queries from natural language instructions.
- **FR-DATA-03:** Shall execute queries against Parquet files via DuckDB.
- **FR-DATA-04:** Shall perform confidence-scored relationship discovery:
 - High confidence → auto-join datasets.
 - Low confidence → prompt user for confirmation before joining.
- **FR-DATA-05:** Shall persist query results as Artifacts.
- **FR-DATA-06:** Shall support multi-step analytical workflows.

5.5.3 Visualization Agent

- **FR-VIZ-01:** Shall recommend appropriate chart types based on the result table structure.
- **FR-VIZ-02:** Shall render user-selected charts using Matplotlib.

- **FR-VIZ-03:** Charts shall be persisted and associated with the generating Project.
- **FR-VIZ-04:** The Visualization Agent shall *not* interpret data or draw business conclusions.

5.5.4 Insight Agent

- **FR-INS-01:** Shall detect trends and anomalies in result tables and chart data.
- **FR-INS-02:** Shall compare current results against historical Artifacts.
- **FR-INS-03:** Shall proactively surface insights (e.g. trend detected, anomaly found, key contributor identified, correlation discovered).

5.6 Artifact System

- **FR-ART-01:** Artifacts are *exclusively* generated by agent workflows — no manual creation.
- **FR-ART-02:** Artifacts shall be persistent, searchable, and reusable across sessions.
- **FR-ART-03:** Artifact retrieval shall use **vector/semantic search** to support fuzzy recall.
- **FR-ART-04:** Artifacts shall not support editing, deletion, or versioning.
- **FR-ART-05:** An Artifact always represents the *latest* computed state.

5.7 Execution Trace

- **FR-TRACE-01:** Every query shall produce a user-visible Execution Trace.
- **FR-TRACE-02:** The Execution Trace shall contain: user query, execution plan, agent actions, generated SQL, tools used, artifacts created, and execution results.
- **FR-TRACE-03:** The Execution Trace shall *not* expose chain-of-thought reasoning.

5.8 SQL Visibility & Editability

- **FR-SQL-01:** Generated SQL shall be visible to the user within the Execution Trace.
- **FR-SQL-02:** Users shall be able to edit and re-run generated SQL.
- **FR-SQL-03:** SQL editing is a secondary workflow — natural language remains the primary interface.
- **FR-SQL-04:** SQL dialect shall be DuckDB SQL (PostgreSQL-compatible style where possible).

5.9 Memory Architecture

The system maintains four memory layers within a Project:

Memory Layer	Content
Dataset Memory	Knowledge about uploaded datasets and their schemas.
Artifact Memory	Generated analytical assets, searchable by vector similarity.
Execution Memory	Plans and execution traces from past queries.
Conversation Memory	User interaction history.

5.10 User Journey

1. Register & Login
2. Create a Project
3. Upload Datasets → automated profiling & metadata extraction
4. Ask a natural language question
5. Planner Agent creates an execution plan
6. Data Agent executes analysis → Artifact created
7. Visualization Agent recommends chart types → user selects → chart generated
8. Insight Agent generates proactive observations
9. Execution Trace becomes available for review
10. User asks follow-up questions; prior Artifacts are reused

6 Non-Functional Requirements

Category	Requirement
Performance	Query execution for typical engineering datasets (<10M rows) should complete within 10 seconds for standard aggregations.
Scalability	Object storage (S3/MinIO) and Parquet format must support datasets growing to hundreds of gigabytes without schema changes.
Reliability	Agent failures must trigger automatic re-planning; the system must not return silent errors.
Transparency	All agent actions must be traceable through the Execution Trace UI.
Usability	The interface must feel conversational and exploratory — not like database administration software.
Security	Authentication via email/password with hashed credentials. All dataset access scoped to project ownership.

Category	Requirement
Maintainability	Each agent must be independently deployable and testable via tool-calling interfaces.

7 Technology Stack

Layer	Technology
Frontend	Next.js
Backend	FastAPI (Python)
Metadata Store	PostgreSQL
Analytics Engine	DuckDB
Object Storage	S3-compatible (MinIO / Cloudflare R2 / AWS S3)
Data Format	Parquet (converted from CSV/XLSX at ingest)
Visualisation	Matplotlib
AI Runtime	Multi-Agent Architecture with Tool Calling
LLM	OpenAI-compatible API (OpenAI, OpenRouter, Together, Groq, vLLM)

7.1 Storage Architecture Rationale

- **Parquet + DuckDB:** Chosen over storing datasets in PostgreSQL for columnar analytics performance, efficient aggregations, and cost-effective large-dataset handling.
- **PostgreSQL:** Stores only metadata (users, projects, dataset/column metadata, artifacts metadata, execution traces, charts metadata, conversations) — never analytical data.
- **S3 Object Storage:** Provides scalability, separation of concerns, and lower storage cost versus database blobs.

8 Data Model

Conceptual Entity Relationships

- **User** owns one or more **Projects**
- **Project** contains **Datasets**, **Artifacts**, **Charts**, **Execution Traces**, and exactly one **Conversation**
- **Dataset** has column-level **Metadata**

- **Artifact** is generated by an **Agent Workflow**; linked to one **Project**
- **Chart** is generated from an **Artifact**; linked to one **Project**
- **Execution Trace** records all agent activities for a user query turn
- **Conversation** records the full user interaction history for a Project

9 Tool Calling Architecture

Agents interact exclusively through defined tools, not prompt-only chains.

Agent	Tool	Description
Data Agent	read_dataset()	Read a dataset's Parquet file from S3.
Data Agent	execute_sql()	Run a DuckDB SQL query.
Data Agent	save_artifact()	Persist a result table as an Artifact.
Data Agent	retrieve_metadata()	Fetch column/schema metadata from PostgreSQL.
Viz Agent	generate_chart()	Render a Matplotlib chart from a result table.
Viz Agent	save_chart()	Persist a rendered chart.
Insight Agent	detect_anomaly()	Analyse a result table for anomalies.
Insight Agent	summarise_results()	Generate natural language observations.

10 Goals & Non-Goals

10.1 Goals (MVP)

- Conversational Analytics via natural language
- Multi-Agent Orchestration (Planner, Data, Visualization, Insight)
- Cross-Dataset Analysis with runtime relationship discovery
- Persistent Knowledge Workspace with Artifact memory
- Execution Transparency through user-visible traces
- Artifact Reuse via semantic vector search
- Vibe Analytics — low-friction, curiosity-first UX

10.2 Non-Goals (Explicitly Excluded from MVP)

- Dashboard Builder
- Data Editing or Deletion
- Real-Time Analytics
- Artifact Versioning
- Report Export
- Manual Artifact Creation
- Multiple Conversations per Project
- Machine Learning Model Training
- Enterprise Collaboration / Shared Workspaces
- Role-Based Access Control (RBAC)
- Data Warehouse Connectivity
- OAuth / SSO Authentication

11 Key Differentiators

- **Vibe Analytics** — feel of ChatGPT, power of a structured analytics backend
- **Agentic Knowledge Workspace** — memory persists across all sessions in a Project
- **Persistent Artifact Memory** — every analysis becomes reusable knowledge
- **Cross-Dataset Reasoning** — LLM-driven schema inference with confidence-based joining
- **Transparent Execution Trace** — users see exactly what happened, building trust
- **Artifact Reuse via Semantic Search** — fuzzy recall of prior analyses by intent
- **Closer to Cursor/NotebookLM than Tableau/Power BI**

12 Success Criteria

A successful MVP user session shall allow an Engineering Analyst to:

1. Upload one or more engineering datasets without configuration.
2. Ask natural language questions and receive accurate, grounded answers.
3. Generate charts from analytical results with a single selection.
4. Receive proactive insights without explicitly requesting them.
5. Understand what the system did via the Execution Trace.
6. Continue analysis across sessions, reusing prior Artifacts.

All of the above **without requiring SQL expertise**.

13 Future Expansion (Out of Scope)

The following capabilities are candidates for future product iterations and must not influence MVP requirements:

Dashboard Builder, Multi-User Collaboration, Shared Projects, RBAC, Artifact Versioning, Scheduled Analysis, Real-Time Data, External Data Connectors, ML Workflows, Report Export, Dashboard Publishing.

Document History

Version	Date	Author	Notes
1.0	May 31, 2026	Product Team	Initial draft from SRS discovery summary.